

AMIHACKS

This packet presents three full-length, in-depth problem statements proposed for the AmiHacks hackathon. Each statement is deliberately kept comprehensive at this stage so the faculty panel can evaluate the complete scope, difficulty, and intent behind every track. Once approved, these will be condensed into participant-facing briefs for the event itself.

TRACK A	TRACK B	TRACK C
NGO / Social Impact	Industry / Open Innovation	Industry / Deep-Tech
Surplus-to-Shelter Real-Time Food Rescue Routing	CityPulse The Live Civic Health Dashboard	SentinelAPI Zero-Trust API Vulnerability Scanner

Event AmiHacks

Document status Draft -- Full-length version for approval (to be trimmed post-approval)

Number of tracks 3 (1 NGO/social-impact track, 2 industry/open-innovation tracks)

Suggested duration 24 hours

Team skill level Mixed -- beginner-friendly entry point with an advanced ceiling in every track

Prepared by the AmiHacks organizing team for internal faculty circulation. Not for public distribution in this form.

Contents

- Track A -- Surplus-to-Shelter: Real-Time Food Rescue Routing (NGO / Social Impact)
- Track B -- CityPulse: The Live Civic Health Dashboard (Industry / Open Innovation)
- Track C -- SentinelAPI: Zero-Trust API Vulnerability Scanner (Industry / Deep-Tech)
- Appendix -- Comparative Summary & Approval Checklist

Problem Statement-2: CityPulse: The Live Civic Health Dashboard

"One glance should tell a resident what's really happening in their neighborhood -- and why it matters."

Domain	Civic data fusion, smart-cities analytics, and public-facing visualization
Difficulty	Intermediate-Advanced -- low floor to start, high ceiling for correlation/ML work
Duration	24 hours Tech stack: open / any
Focus	Industry / open-innovation, with clear public-good relevance

1. Problem Statement

Local governments and residents rarely have a single, real-time view of how a neighborhood is actually doing. The data exists -- traffic incidents, public transit delays, air quality readings, noise complaints, power outages, weather alerts -- but it is scattered across a dozen disconnected feeds and dashboards. Residents find out about problems (a flooded underpass, a gas leak, a transit outage) only after they are stuck in it. City staff cannot spot emerging patterns (three outages on the same block, complaints clustering near a school) until someone escalates a complaint manually. The difficulty is not collecting one data source -- it is fusing multiple noisy, differently formatted, differently paced feeds into something a non-technical person can glance at and understand in seconds.

Why it is hard

Civic feeds arrive at wildly different rates and formats -- a weather alert every few hours, a transit delay every few minutes, a 311 complaint at any moment -- and rarely share a common schema, identifier, or timestamp convention. Turning that mismatch into a single trustworthy pulse requires real data-normalization work, not just a shared map view; and surfacing genuine correlations (rather than coincidences) without misleading a non-technical viewer is a real analytical and communication challenge.

2. Target Users / Stakeholders

- Primary: Residents of a neighborhood or city district.
- Secondary: City ops staff, local journalists, emergency responders, and small business owners deciding whether to open today.

3. Current Situation & Pain Points

- Data lives in silos (transit app, weather app, city 311 portal, utility outage map).
- No correlation between feeds -- a resident won't know a "power outage" and a "traffic light down" three blocks apart are related.
- Alerts are reactive, not predictive.
- Existing dashboards are built for analysts, not the general public.

4. Objective

Build a system that ingests multiple live or simulated civic data streams and produces a single, understandable pulse of an area -- surfacing anomalies and correlations a human would otherwise miss. The specific feeds

chosen, the anomaly-detection technique, and the visual metaphor for pulse are all open to each team's interpretation.

5. Expected Solution Capabilities

- 1 Ingest 3+ distinct data types (e.g., weather, transit, incident reports, air quality, social sentiment).
- 2 Normalize and timestamp mismatched feeds into a common data model.
- 3 Detect basic anomalies or correlations (e.g., a spike in complaints coinciding with a weather event).
- 4 Present a live, glanceable visual dashboard or map.
- 5 Generate a plain-language summary -- what's happening right now and why it matters.
- 6 Optional: alerting/notification for a chosen threshold.
- 7 Optional: historical replay to demonstrate pattern-detection working over past data.

6. Innovation Opportunities

- AI/ML: anomaly detection and time-series correlation across heterogeneous feeds.
- NLP: summarizing raw feeds into a human-readable, plain-language narrative.
- Agentic AI: an agent that continuously monitors feeds and proactively raises flags.
- Data analytics/visualization: creative, non-dashboard-cliche ways to show pulse (e.g., a literal heartbeat/pulse metaphor, a heat-map timeline, or a narrative ticker).
- Geospatial/IoT: simulated sensor data layered onto a map.

As with the other tracks, breadth of feeds matters less than the quality of the fusion and the clarity of the resulting summary -- three well-correlated feeds beat six that are never actually connected.

7. Constraints & Considerations

- Must work with public or synthetic data -- no proprietary city access is assumed.
- Should degrade gracefully if one feed is missing or delayed.
- Privacy: no reliance on identifying individuals from social/complaint data.
- Should be understandable by a non-technical viewer within roughly 10 seconds.
- Epistemic honesty: correlations shown to residents should be clearly framed as possible links, not confirmed causes, to avoid spreading civic misinformation.

8. Hackathon Scope (24 Hours)

A working MVP is realistic within the timeframe:

- At least three simulated or public feeds (e.g., weather, transit delays, and 311-style incident reports) normalized into one schema.
- A simple anomaly/correlation rule (e.g., complaint density rising alongside a weather alert in the same zone) computed on a rolling window.
- A live map or dashboard view that a non-technical person can read at a glance.
- A short, plain-language summary line generated from the current state of the feeds (template-based or LLM-assisted, grounded strictly in the ingested data).
- Advanced teams can add true ML-based anomaly detection, an agentic monitoring layer, threshold-based alerting, or historical replay of a simulated multi-day dataset.